



- Querying, Explaining, and Steering **World Models** through Language

Amitava Das[★]

[★]Pragya Lab, BITS Pilani Goa, India

Abstract

World models are increasingly viewed as foundations for agents, robotics, and physical AI, yet they remain largely *silent simulators*: they can forecast futures, but users cannot easily ask *why*, explore *what if*, inspect uncertainty, or steer rollouts through language. We introduce **YONDER: Querying, Explaining, and Steering World Models Through Language**, a modular framework that turns predictive world models into *language-addressable simulators*. Rather than treating language as post-hoc narration, YONDER formalizes natural-language instructions as executable *world operations* over predictive latent states.

Our implementation uses a frozen **V-JEPA-style video world encoder** as the predictive backbone, a lightweight **world-language adapter** to convert rollout features, trajectory estimates, occupancy maps, and uncertainty signals into language-addressable tokens, and an instruction-tuned **Llama-3.1-8B-Instruct** interface for natural-language interaction. The world model is kept frozen; only the adapter and lightweight steering heads are trained. This avoids expensive model merging while preserving compatibility with future backends such as Cosmos, Genie-style video simulators, trajectory forecasters, and embodied planners.

YONDER supports six classes of language-grounded world operations: Query, Explain, Intervene, Steer, Plan, and Inspect. Given a short video, users can ask questions such as *What happens next?*, *Where is the hidden cyclist?*, *Why do you predict braking?*, *What if the pedestrian runs?*, or *Generate the safest future*. YONDER returns grounded text answers together with future trajectory overlays, occupancy heatmaps, uncertainty maps, counterfactual comparisons, and planning recommendations.

We introduce **YONDER-Bench**, an evaluation suite for language-grounded world-model interaction over curated 10-second traffic and urban-scene videos containing pedestrians, cyclists, vehicles, occlusions, dense interactions, and near-risk events. The benchmark measures **future-answer correctness**, **rollout groundedness**, **counterfactual consistency**, **explanation faithfulness**, **uncertainty calibration**, **steering success**, and **planning utility**. Compared with frame-only video-language baselines, YONDER is designed to ground answers in predictive rollouts rather than static visual evidence, enabling evaluation of whether language faithfully accesses and manipulates the model's simulated futures.

In our first release, we provide **YONDER-Demo**, a public Hugging Face interface where users upload a 10-second video and interact with a predictive world model through natural language. The release includes demo code, instruction taxonomy, curated evaluation videos, YONDER-Bench protocol, adapter checkpoints, and leaderboard infrastructure. YONDER reframes world models from passive prediction engines into **interactive reasoning systems**: models that not only predict what happens next, but also explain why, explore alternatives, expose uncertainty, and allow humans to steer imagined futures through language.

1 World Models: From Prediction to Interaction

The central promise of a world model is not recognition, but **anticipation**. A vision model may identify a cyclist, a pedestrian, or a vehicle; a world model must estimate where they may go, how they may interact, what may remain hidden, and which futures become possible. This changes the object of modeling from a static observation to a **space of futures** (Ha and Schmidhuber, 2018; LeCun, 2022; Hafner et al., 2023).

Recent model families instantiate this shift in different forms. **Latent predictive models** such as V-JEPA and V-JEPA 2 predict future representations rather than pixels (Bardes et al., 2024; Assran et al., 2025); **action-conditioned models** such as Dreamer roll latent dynamics forward under candidate actions (Hafner et al., 2023); **interactive simulators** such as Genie and **world foundation models** such

as Cosmos treat learned dynamics as reusable substrates for controllable environments, robotics, and physical AI (Bruce et al., 2024; Agarwal et al., 2025). Across these approaches, the model output is no longer only a label, caption, or segmentation mask, but a rollout: a predicted trajectory, occupancy field, latent future, or action-conditioned simulation.

Evaluation has followed this predictive turn. Benchmarks now measure future-state accuracy, temporal consistency, latent retrieval, planning success, and rollout fidelity (Hafner et al., 2023; Bardes et al., 2024; Assran et al., 2025). These are necessary tests of whether a model can simulate. But they leave untouched a second question: **Can a human interact with that simulation?**

A rollout may encode where an occluded cyclist is likely to emerge, why a vehicle is expected to brake, which actor drives uncertainty, or which action reduces risk. Yet standard evaluation treats such information as an output to be scored, not a future to be queried, explained, modified, or steered. Thus the field has made world models increasingly predictive, but not yet **interactive**.

1.1 The Interaction Gap

World models can simulate futures, but they remain largely silent. A model may internally represent where an occluded cyclist will emerge, which actor is most uncertain, or which trajectory minimizes risk. Yet these beliefs remain trapped inside rollouts, latent states, and occupancy maps.

This is the **interaction gap: world models can imagine futures, but humans cannot naturally converse with those imagined futures.**

The gap becomes particularly acute in embodied settings. In autonomous driving, users may ask: *Where will the cyclist emerge?* In robotics: *What if the object slips?* In planning: *Which action minimizes risk?* These are not conventional video-understanding questions; they are interactions with predictive simulations.

1.2 Language as an Interface to Predictive Simulation

Language has emerged as the dominant interface for interacting with complex models (Li et al., 2023a,b; Maaz et al., 2023). Yet most video-language systems make videos *describable*, not simulations *addressable*. They answer questions about observed frames rather than predicted futures.

We argue that language should operate directly on predictive computation. Questions such as *Where will the cyclist be in two seconds?*, *Why are you uncertain?*, or *What if the pedestrian starts running?* should access, explain, and modify the underlying simulation itself.

This view reframes language from a captioning mechanism into an **execution interface for predictive simulation**. In this setting, **prediction is the substrate; interaction is the interface.**

We formalize these interactions as **World Operations**: executable language-grounded operations over predictive simulations.

- **Query**: retrieve information about future states.
- **Explain**: justify predictions and uncertainty.
- **Intervene**: modify actors, events, or environments and re-simulate.
- **Steer**: constrain rollouts toward desired objectives.
- **Plan**: derive actions from simulated futures.
- **Inspect**: expose latent assumptions, beliefs, and uncertainty.

Together, these operations define the missing interface between predictive world models and human intent. YONDER builds on this premise: world models should not only generate futures, but make those futures **queryable, explainable, steerable, plannable, and inspectable through language.**

2 YONDER: Language-Guided Interaction with Predictive Simulations

YONDER instantiates an **interface-level fusion** between a predictive world model and a language model. Instead of merging model weights or training a monolithic video-language simulator, we keep the world model frozen and train only the modules needed to make its predictive states language-addressable. This follows the efficiency principle of frozen-encoder multimodal systems (Li et al., 2023a; Maaz et al., 2023), but shifts the target from *observed video content* to **future-oriented predictive simulation**.

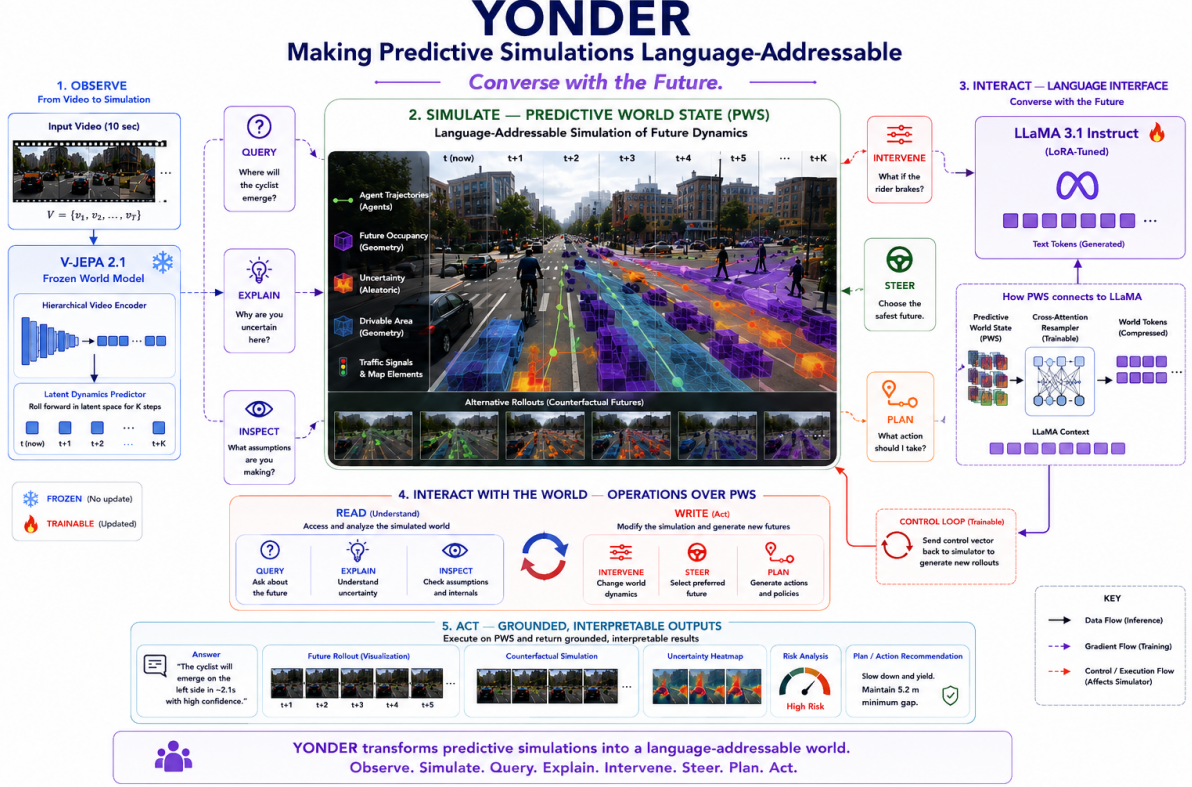


Figure 1: **YONDER: Language-Guided Interaction with Predictive Simulations**. Starting from a short input video, a frozen **V-JEPA 2.1** world model constructs a **Predictive World State (PWS)**, a language-addressable simulation containing future trajectories, occupancy forecasts, uncertainty fields, drivable regions, traffic semantics, and alternative rollouts. Unlike conventional video-language systems that describe observed content, YONDER treats the predictive simulation itself as the object of interaction. Through a **LoRA-tuned LLaMA 3.1** interface, users can execute six classes of **World Operations** over the PWS. **Read operations** (Query, Explain, Inspect) access future predictions, uncertainty estimates, and latent assumptions without modifying the simulation. **Write operations** (Intervene, Steer, Plan) generate control signals that alter simulation dynamics, produce counterfactual futures, and enable goal-directed reasoning. A trainable world-language interface converts predictive world states into compact world tokens that can be consumed by the language model, while a control loop feeds language-derived interventions back into the simulator to generate updated rollouts. The system returns grounded and interpretable outputs including future-state predictions, explanations, counterfactual simulations, uncertainty visualizations, risk assessments, and action recommendations. By transforming predictive simulations into a language-addressable substrate, YONDER shifts world models from passive future prediction to interactive future reasoning, enabling users to **observe, simulate, query, explain, intervene, steer, plan, and act** directly over imagined futures.

Concretely, YONDER uses a frozen **V-JEPA 2.1**-style world backbone to encode a short video and produce future-oriented latent dynamics (Assran et al., 2025). These outputs are organized into a structured **Predictive World State (PWS)**:

$$\mathcal{P} = \{Z, T, O, U, R, A\},$$

where Z denotes future latent states, T agent trajectories, O occupancy maps, U uncertainty fields, R action-conditioned rollouts, and A attention or saliency regions. Thus, $\mathcal{P}$ is not a caption of the video; it is a compact representation of the futures the world model considers possible.

A trainable **World–Language Interface** maps this predictive state into language-consumable world tokens:

$$H = A_\phi(\mathcal{P}),$$

where A_ϕ consists of a PWS encoder, cross-attention resampler, and projection layer. The resulting world tokens H are concatenated with a user instruction I and passed to a **LoRA-tuned LLaMA interface**:

$$Y = L_\theta([H; I]).$$

Here, LoRA provides parameter-efficient adaptation of the language model while leaving the base model largely intact (Hu et al., 2021).

The language output Y is then decoded into an executable world operation and its parameters:

$$(o, \pi) = D_\psi(Y),$$

where $o \in \{\text{Query, Explain, Inspect, Intervene, Steer, Plan}\}$. **Read operations** such as Query, Explain, and Inspect access the PWS without changing the simulation. **Write operations** such as Intervene, Steer, and Plan generate control vectors through trainable control heads:

$$c = g_\omega(o, \pi, \mathcal{P}),$$

which are fed back into the simulator to produce an updated predictive world state:

$$\mathcal{P}' = f_{\text{WM}}(\mathcal{P}, c).$$

In this architecture, the language model does not merely describe the video. It operates over the predictive world state. YONDER therefore performs **language-guided execution over simulated futures**: users can ask what may happen, why the model expects it, what alternatives exist, how outcomes change under intervention, and which actions follow from the resulting futures.

2.1 System Overview

Let $V = \{v_1, v_2, \dots, v_T\}$ denote an input video with T observed frames. YONDER first passes V through a frozen predictive world model f_{WM} to construct a **Predictive World State (PWS)**:

$$\mathcal{P} = f_{\text{WM}}(V).$$

The key design choice is that YONDER does not expose the language model directly to raw video frames. Instead, all language reasoning is grounded in $\mathcal{P}$, a future-oriented representation containing predicted latent states, trajectories, occupancy, uncertainty, and rollouts. Thus, the language model reasons over **what the world model expects to happen**, not merely over **what the video currently shows**.

YONDER converts $\mathcal{P}$ into language-consumable world tokens through a trainable World–Language Interface A_ϕ :

$$H = A_\phi(\mathcal{P}),$$

where $H = \{h_1, \dots, h_M\}$ denotes a compact sequence of world tokens. Given a user instruction $I = \{w_1, \dots, w_L\}$, a LoRA-tuned language model L_θ produces an operation-conditioned output:

$$Y = L_\theta([H; I]).$$

The output Y is decoded into a world operation o and operation parameters π :

$$(o, \pi) = D_\psi(Y),$$

where

$$o \in \{\text{Query, Explain, Inspect, Intervene, Steer, Plan}\}.$$

For **read operations** such as Query, Explain, and Inspect, YONDER retrieves information from $\mathcal{P}$ and returns grounded textual or visual outputs. For **write operations** such as Intervene, Steer, and Plan, YONDER generates a control signal

$$c = g_{\omega}(o, \pi, \mathcal{P}),$$

which is fed back into the simulator to obtain an updated predictive world state:

$$\mathcal{P}' = f_{\text{WM}}(\mathcal{P}, c).$$

The complete computation can therefore be summarized as

$$V \rightarrow \mathcal{P} \rightarrow H \rightarrow Y \rightarrow (o, \pi) \rightarrow c \rightarrow \mathcal{P}'.$$

This formulation turns future prediction into an **interactive simulation loop**: the world model imagines futures, the language interface makes those futures addressable, and user instructions can either read from the simulation or modify it to produce new futures.

2.2 Predictive World State

The central abstraction in YONDER is the **Predictive World State (PWS)**. A world model does not produce a single future; it produces a structured set of latent expectations about how the scene may evolve. YONDER organizes these expectations into a unified representation that can be accessed by language.

Formally, we define the Predictive World State as

$$\mathcal{P} = \{Z, T, O, U, R, A\},$$

where Z denotes future latent states, T agent trajectories, O occupancy forecasts, U uncertainty fields, R action-conditioned rollouts, and A attention or saliency regions.

The latent component

$$Z = \{z_{t+1}, z_{t+2}, \dots, z_{t+K}\}$$

captures the simulator’s predicted future world states over a horizon of K steps. These latent states preserve high-level scene dynamics without requiring pixel-level reconstruction at every step.

The trajectory component

$$T = \{\tau_1, \tau_2, \dots, \tau_N\}$$

represents predicted motion for N agents. Each trajectory is defined as

$$\tau_i = \{(x_{i,t+1}, y_{i,t+1}), \dots, (x_{i,t+K}, y_{i,t+K})\},$$

allowing YONDER to reason about future paths, crossings, yielding behavior, and possible collisions.

The occupancy component

$$O = \{o_{t+1}, o_{t+2}, \dots, o_{t+K}\}$$

encodes where agents or objects are likely to occupy space in future frames. Occupancy provides a spatial interface for questions such as *Where will the cyclist emerge?* or *Which region is risky?*

The uncertainty component

$$U = \{u_{t+1}, u_{t+2}, \dots, u_{t+K}\}$$

captures ambiguity over future outcomes. It allows YONDER to expose whether a prediction is confident, multimodal, occlusion-driven, or sensitive to small changes in the scene.

The rollout component

$$R = \{r^{(1)}, r^{(2)}, \dots, r^{(M)}\}$$

stores alternative futures produced under different assumptions, samples, or interventions. These rollouts support counterfactual and steering queries, such as *What if the pedestrian starts running?* or *Generate the safest future.*

Finally, the attention component A captures visual or latent regions that contribute to the predicted future. This enables explanation-oriented operations by linking predicted outcomes to supporting evidence in the scene.

Thus, the PWS is not a raw hidden state and not a caption. It is a **language-addressable predictive memory**: a structured representation of future latents, agent motion, spatial occupancy, uncertainty, alternative rollouts, and explanatory evidence. All YONDER operations act on this state, making predictive simulation queryable, explainable, steerable, and inspectable through language.

2.3 World-Language Interface

The **Predictive World State** is structured, but it is not naturally linguistic. Its components live in different spaces: latent tensors for future states, coordinate sequences for trajectories, spatial grids for occupancy, uncertainty fields for ambiguity, rollout graphs for alternative futures, and saliency maps for evidence. A language model cannot directly consume these heterogeneous objects without a bridge.

YONDER therefore introduces a trainable **World-Language Interface** that converts the PWS into a compact sequence of **world tokens**. These tokens act as the language model’s view of the predictive simulation: they preserve future-relevant structure while matching the token interface expected by an instruction-tuned LLM.

Let

$$\mathcal{P} = \{Z, T, O, U, R, A\}$$

denote the Predictive World State. Each component $P_i \in \mathcal{P}$ is first encoded by a component-specific encoder ϕ_i :

$$e_i = \phi_i(P_i), \quad P_i \in \{Z, T, O, U, R, A\}.$$

Here, each encoder is chosen to respect the structure of its input. Latent states are projected through temporal pooling or attention; trajectories are encoded as ordered coordinate sequences; occupancy and uncertainty maps are encoded as spatial grids; rollout sets are encoded as multimodal future branches; and attention maps are encoded as evidence summaries. This yields a set of aligned component embeddings:

$$E = [e_Z; e_T; e_O; e_U; e_R; e_A].$$

The concatenated representation E may still be too large and redundant for direct insertion into the LLM context. YONDER therefore applies a trainable cross-attention resampler. Given M learned query vectors

$$Q = \{q_1, q_2, \dots, q_M\},$$

the resampler attends over E and extracts a fixed-size set of world tokens:

$$H = \text{Resampler}(Q, E),$$

where

$$H = \{h_1, h_2, \dots, h_M\}.$$

Each world token h_m is computed as an attention-weighted summary of predictive information:

$$h_m = \sum_j \alpha_{mj} W_v e_j, \quad \alpha_{mj} = \text{softmax}_j \left(\frac{(W_q q_m)^\top (W_k e_j)}{\sqrt{d}} \right).$$

The world tokens are then projected into the LLM embedding space:

$$\tilde{H} = W_p H,$$

and concatenated with the user instruction tokens I :

$$X = [\tilde{H}; I].$$

The language model receives X as its context:

$$Y = L_\theta(X),$$

where Y is later decoded into either a grounded response or an executable world operation.

This design makes the connection between the world model and the language model explicit. The LLM does not reason over raw video frames, nor over unstructured latent tensors. It reasons over **world tokens**: compressed, language-aligned summaries of future latents, trajectories, occupancy, uncertainty, rollouts, and evidence. Thus, the World–Language Interface is the mechanism that makes predictive simulation **language-addressable**.

2.4 Instruction-Grounded World Operations

Once the Predictive World State has been converted into world tokens, YONDER treats a user instruction as a request to execute an operation over the simulated future. Let

$$I = \{w_1, w_2, \dots, w_L\}$$

denote the instruction tokens and let $H = \{h_1, \dots, h_M\}$ denote the world tokens produced by the World–Language Interface.

The language model receives both streams as context:

$$Y = L_\theta([H; I]),$$

where Y contains the model’s operation-conditioned response. Unlike ordinary video-language generation, Y is not interpreted only as free-form text. YONDER decodes it into an executable world operation and its arguments:

$$(o, \pi) = D_\psi(Y),$$

where o denotes the operation type and π denotes operation-specific parameters such as actor identity, time horizon, target region, intervention type, or optimization objective.

We define the operation space as

$$\mathcal{O} = \{\text{Query, Explain, Inspect, Intervene, Steer, Plan}\}.$$

These operations form the interface through which language acts on predictive simulation.

YONDER separates operations into two groups. **Read operations** access the Predictive World State without changing it:

$$\mathcal{O}_R = \{\text{Query, Explain, Inspect}\}.$$

Query retrieves future-state information such as predicted positions, trajectories, occupancy, or likelihoods. Explain identifies evidence, uncertainty sources, or latent factors supporting a prediction. Inspect exposes internal assumptions, ambiguity, and belief-like priors encoded in the PWS.

Write operations modify the predictive simulation:

$$\mathcal{O}_W = \{\text{Intervene, Steer, Plan}\}.$$

Intervene changes a condition in the simulated world, such as an actor speed, object state, or environmental context. Steer constrains the rollout toward an objective, such as safety, caution, or worst-case risk. Plan searches over possible future outcomes to recommend actions.

Thus, a natural-language instruction is transformed from a passive prompt into an executable operation:

$$I \longrightarrow (o, \pi) \longrightarrow \text{Exec}(o, \pi, \mathcal{P}).$$

For read operations, execution returns a grounded response:

$$Y_{\text{read}} = \text{Read}(o, \pi, \mathcal{P}).$$

For write operations, execution produces a modified predictive world state:

$$\mathcal{P}' = \text{Write}(o, \pi, \mathcal{P}).$$

This distinction is central to YONDER. **Read operations make simulated futures accessible; write operations make simulated futures controllable.** Together, they turn a predictive world model from a passive rollout generator into an interactive system whose imagined futures can be queried, explained, inspected, altered, steered, and used for planning through language.

2.5 Control and Simulation Steering

Write operations require more than textual generation. When a user asks *What if the cyclist brakes?*, *Generate the safest future*, or *Which action should the ego vehicle take?*, YONDER must modify the predictive simulation itself. We therefore introduce trainable **Control Heads** that convert decoded world operations into continuous control signals.

Given an operation o , its parameters π , and the current Predictive World State $\mathcal{P}$, the control head produces

$$c = g_{\omega}(o, \pi, \mathcal{P}),$$

where c encodes the intervention or steering command to be applied to the simulator. Depending on the operation, c may represent an actor-level perturbation, a trajectory constraint, a safety objective, a target region, or an action prior.

The control signal is then injected back into the predictive simulator to obtain an updated world state:

$$\mathcal{P}' = f_{\text{WM}}(\mathcal{P}, c).$$

Here, $\mathcal{P}'$ is not a textual description of a counterfactual; it is a newly simulated Predictive World State containing revised future latents, trajectories, occupancy, uncertainty, and rollouts.

This closed-loop mechanism separates YONDER from ordinary video-language models. A standard model may answer *the cyclist might slow down*; YONDER instead alters the simulated dynamics, re-generates the future, and grounds its answer in the updated rollout. Thus, language does not merely describe a hypothetical future; it **executes** a change in predictive simulation space.

For an intervention instruction such as

What if the cyclist brakes?

the operation decoder identifies the target actor, action, and temporal window. The control head then produces a braking intervention c_{brake} , which modifies the cyclist’s future dynamics and triggers a new rollout:

$$\mathcal{P}_{\text{brake}} = f_{\text{WM}}(\mathcal{P}, c_{\text{brake}}).$$

Subsequent reasoning is performed over $\mathcal{P}_{\text{brake}}$, allowing YONDER to compare the original and intervened futures.

For a steering instruction such as

Generate the safest future,

the control signal is instead optimized toward a criterion such as collision avoidance, uncertainty reduction, or minimum-risk trajectory selection:

$$c^* = \arg \min_c \mathcal{J}_{\text{risk}}(f_{\text{WM}}(\mathcal{P}, c)),$$

where $\mathcal{J}_{\text{risk}}$ measures risk over the resulting predictive rollout. The corresponding updated state

$$\mathcal{P}^* = f_{\text{WM}}(\mathcal{P}, c^*)$$

serves as the basis for the final response or plan.

For planning, YONDER evaluates candidate controls

$$\mathcal{C} = \{c_1, c_2, \dots, c_M\}$$

by rolling each one forward:

$$\mathcal{P}^{(m)} = f_{\text{WM}}(\mathcal{P}, c_m).$$

The selected action is obtained by minimizing a task-dependent objective:

$$m^* = \arg \min_m \mathcal{J}(\mathcal{P}^{(m)}), \quad c^* = c_{m^*}.$$

Thus, YONDER implements a unified mechanism for **intervention**, **steering**, and **planning**: language is decoded into an operation, the operation is converted into a control signal, the simulator produces an updated future, and the language model reasons over the revised Predictive World State. This converts predictive simulation from a passive forecasting process into an **executable substrate for language-guided reasoning and action**.

2.6 Training Objectives

YONDER is trained as an **interface-level fusion** system. The predictive world model remains frozen throughout training. We optimize only four lightweight components: the **World-Language Interface** A_ϕ , the **Operation Decoder** D_ψ , the **Control Heads** g_ω , and the **LoRA parameters** of the language model L_θ . This preserves the predictive structure of the world model while learning how to expose, verbalize, and manipulate its simulated futures through language.

Each training example is represented as

$$(V, I, o^*, \pi^*, Y^*, \mathcal{P}^*),$$

where V is the input video, I is the user instruction, o^* is the target world operation, π^* denotes operation parameters, Y^* is the target textual response, and $\mathcal{P}^*$ is the target predictive world state or rollout when supervision is available.

The frozen world model first constructs

$$\mathcal{P} = f_{\text{WM}}(V),$$

and the World-Language Interface maps it into world tokens:

$$H = A_\phi(\mathcal{P}).$$

The language model then receives world tokens and instruction tokens:

$$Y = L_\theta([H; I]).$$

Finally, the Operation Decoder predicts

$$(\hat{o}, \hat{\pi}) = D_\psi(Y),$$

and, for write operations, the Control Head produces

$$\hat{c} = g_\omega(\hat{o}, \hat{\pi}, \mathcal{P}).$$

The full YONDER objective is

$$\mathcal{L}_{\text{YONDER}} = \mathcal{L}_{\text{inst}} + \lambda_1 \mathcal{L}_{\text{align}} + \lambda_2 \mathcal{L}_{\text{op}} + \lambda_3 \mathcal{L}_{\text{ground}} + \lambda_4 \mathcal{L}_{\text{steer}} + \lambda_5 \mathcal{L}_{\text{calib}}.$$

Instruction loss. The instruction-following objective trains the LoRA-tuned language model to produce the target response conditioned on both the instruction and world tokens:

$$\mathcal{L}_{\text{inst}} = - \sum_{\ell=1}^{|Y^*|} \log p_\theta(y_\ell^* | y_{<\ell}^*, H, I).$$

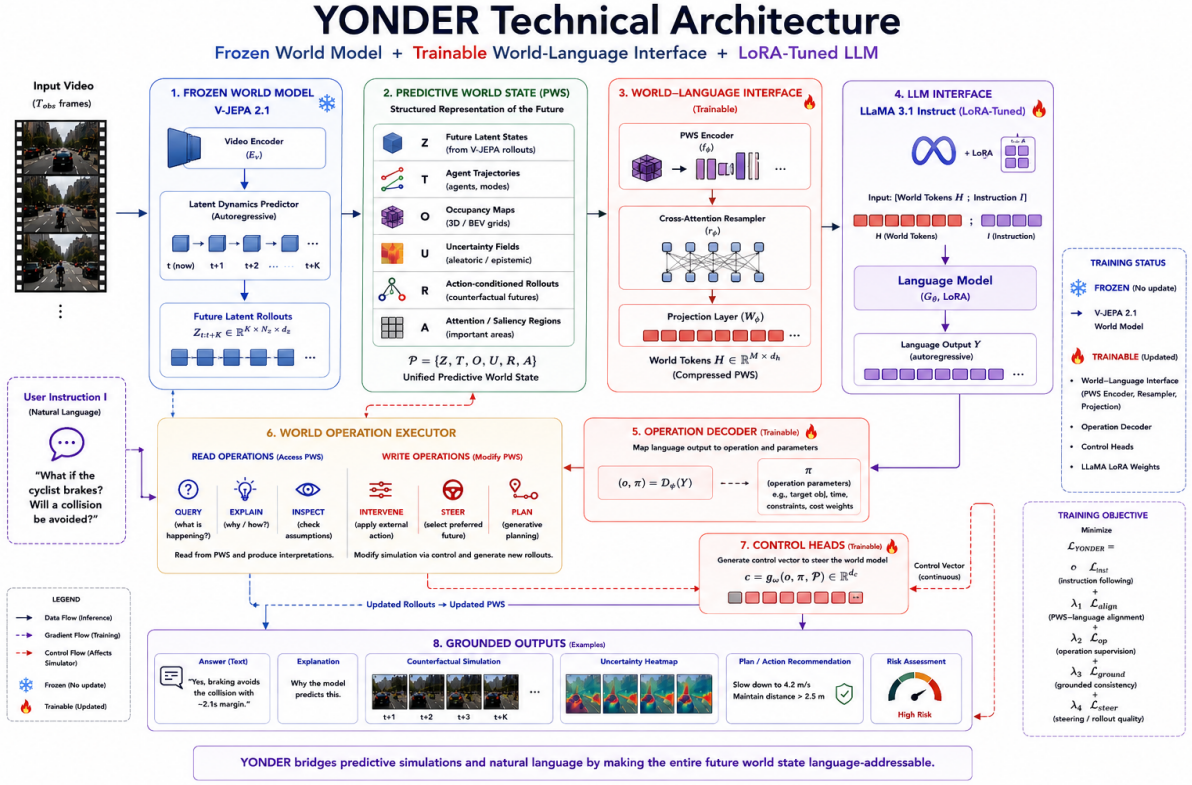


Figure 2: **YONDER Technical Architecture**. YONDER connects a frozen predictive world model and a language model through a trainable **World-Language Interface**, enabling natural-language interaction with simulated futures. Given an input video sequence, a frozen **V-JEPA 2.1** backbone encodes observations and rolls latent dynamics forward to construct a structured **Predictive World State (PWS)**. The PWS serves as the central representation of future simulation and contains future latent states (Z), agent trajectories (T), occupancy maps (O), uncertainty fields (U), action-conditioned rollouts (R), and attention regions (A), summarized as $\mathcal{P} = \{Z, T, O, U, R, A\}$. The trainable World-Language Interface transforms the PWS into compact *world tokens* through a PWS encoder, cross-attention resampler, and projection module. These tokens are combined with user instructions and processed by a **LoRA-tuned LLaMA 3.1** model, whose outputs are decoded into executable world operations and parameters. YONDER distinguishes between **Read Operations** (Query, Explain, Inspect), which access and analyze predictive simulations, and **Write Operations** (Intervene, Steer, Plan), which modify simulations through trainable control vectors and generate updated rollouts. The system produces grounded outputs including future predictions, explanations, counterfactual simulations, uncertainty maps, risk assessments, and action recommendations. Only the World-Language Interface, Operation Decoder, Control Heads, and LoRA parameters are trained, while the underlying V-JEPA world model remains frozen. By introducing a language-addressable predictive world state and executable world operations, YONDER transforms world models from passive future predictors into interactive simulation engines capable of **querying, explaining, inspecting, intervening, steering, and planning** over imagined futures.

This term ensures that YONDER can answer language queries while grounding the response in the predictive world state rather than raw video alone.

World-language alignment loss. The alignment objective forces world tokens to preserve the predictive content of the PWS. Let $s(\cdot)$ denote a projection into a shared contrastive space. For a matched pair $(H, \mathcal{P})$, we maximize similarity, while pushing apart mismatched predictive states $\mathcal{P}^-$ from other videos:

$$\mathcal{L}_{align} = -\log \frac{\exp(\text{sim}(s(H), s(\mathcal{P}))/\tau)}{\exp(\text{sim}(s(H), s(\mathcal{P}))/\tau) + \sum_{\mathcal{P}^-} \exp(\text{sim}(s(H), s(\mathcal{P}^-))/\tau)}.$$

This prevents the adapter from becoming a generic visual summarizer. It must encode the specific future latents, trajectories, occupancy, uncertainty, and rollouts produced by the world model.

Operation supervision loss. YONDER must identify which world operation a user instruction requires.

We supervise the predicted operation $\hat{o}$ with a cross-entropy loss:

$$\mathcal{L}_{\text{op}}^{\text{type}} = -\log p_{\psi}(o^* | Y, H, I).$$

For operation parameters, such as actor identity, time horizon, target region, intervention type, or planning objective, we use a mixed discrete-continuous loss:

$$\mathcal{L}_{\text{op}}^{\text{param}} = \mathcal{L}_{\text{CE}}(\hat{\pi}_d, \pi_d^*) + \|\hat{\pi}_c - \pi_c^*\|_2^2,$$

where π_d denotes discrete parameters and π_c denotes continuous parameters. The full operation loss is

$$\mathcal{L}_{\text{op}} = \mathcal{L}_{\text{op}}^{\text{type}} + \beta \mathcal{L}_{\text{op}}^{\text{param}}.$$

Grounding loss. A response is useful only if it is faithful to the predictive simulation. For read operations, we extract a structured answer $a(\mathcal{P}, o, \pi)$ from the PWS and require the model response to match it:

$$\mathcal{L}_{\text{ground}} = d(\hat{a}, a(\mathcal{P}, o^*, \pi^*)),$$

where $d(\cdot, \cdot)$ is chosen according to the answer type. For categorical answers, d is cross-entropy; for spatial answers, it is an occupancy or coordinate error; for trajectory answers, it is average displacement error; and for uncertainty answers, it is a calibration-aware distance.

For trajectory grounding, we use

$$\mathcal{L}_{\text{traj}} = \frac{1}{NK} \sum_{i=1}^N \sum_{k=1}^K \|\hat{\tau}_{i,t+k} - \tau_{i,t+k}\|_2^2.$$

For occupancy grounding, we use

$$\mathcal{L}_{\text{occ}} = -\sum_{k=1}^K \sum_{x,y} \left[O_{t+k}(x, y) \log \hat{O}_{t+k}(x, y) + (1 - O_{t+k}(x, y)) \log(1 - \hat{O}_{t+k}(x, y)) \right].$$

The grounding objective can therefore be written as

$$\mathcal{L}_{\text{ground}} = \gamma_1 \mathcal{L}_{\text{traj}} + \gamma_2 \mathcal{L}_{\text{occ}} + \gamma_3 \mathcal{L}_{\text{text-ground}}.$$

Steering loss. For write operations, the model must generate a control vector that produces the intended change in the simulation. Given a predicted control $\hat{c}$, the simulator generates

$$\hat{\mathcal{P}}' = f_{\text{WM}}(\mathcal{P}, \hat{c}).$$

If a target counterfactual or steered rollout $\mathcal{P}^*$ is available, we minimize rollout discrepancy:

$$\mathcal{L}_{\text{steer}} = D_{\text{rollout}}(\hat{\mathcal{P}}', \mathcal{P}^*).$$

For trajectory-level steering, this becomes

$$D_{\text{rollout}} = \frac{1}{NK} \sum_{i=1}^N \sum_{k=1}^K \|\hat{\tau}'_{i,t+k} - \tau^*_{i,t+k}\|_2^2.$$

For risk-sensitive steering, we optimize the induced future according to a task cost:

$$\mathcal{L}_{\text{steer}} = \mathcal{J}(f_{\text{WM}}(\mathcal{P}, \hat{c})),$$

where $\mathcal{J}$ may encode collision risk, unsafe proximity, excessive uncertainty, or deviation from a target behavior.

Uncertainty calibration loss. Because many YONDER queries concern uncertainty, we explicitly calibrate predicted confidence against rollout variability. Let $\hat{p}$ denote the model’s predicted confidence and q denote empirical correctness or rollout agreement. We use

$$\mathcal{L}_{\text{calib}} = \sum_{b=1}^B \frac{|B_b|}{n} |\text{acc}(B_b) - \text{conf}(B_b)|,$$

where B_b denotes confidence bin b . This penalizes overconfident explanations and encourages uncertainty responses to reflect actual rollout ambiguity.

Training schedule. We train YONDER in three stages. First, we freeze both the world model and the LLM and train only the World–Language Interface using $\mathcal{L}_{\text{align}}$ and reconstruction-style grounding losses. Second, we enable LoRA updates in the language model and train on instruction–response pairs using $\mathcal{L}_{\text{inst}} + \mathcal{L}_{\text{op}} + \mathcal{L}_{\text{ground}}$. Third, we train the Operation Decoder and Control Heads for write operations using $\mathcal{L}_{\text{steer}} + \mathcal{L}_{\text{calib}}$, while keeping the world model frozen.

This staged procedure is critical. Adapter alignment teaches YONDER to read predictive simulations; instruction tuning teaches it to communicate about them; operation supervision teaches it to select the correct executable operation; and steering optimization teaches it to modify the simulation itself. The result is a language interface that does not merely describe world-model outputs, but **learns to access, explain, and control imagined futures**.

3 YONDER-Bench: Benchmarking Language-Guided Interaction with Predictive Simulations

Language-guided interaction with predictive simulations requires supervision grounded in futures. A user may ask where an agent will appear, why a prediction was made, which actor contributes most uncertainty, what would happen under intervention, or which action minimizes risk. Building and evaluating such capabilities requires questions whose answers are anchored in future outcomes. However, for any real-world scene, only one future is observed, making it impossible to directly collect supervision over the full space of possible worlds.

YONDER-Bench addresses this challenge through a **future-first** construction paradigm. Rather than generating questions and searching for answers, we begin with real future observations, detect what actually happens, extract visual and textual answers from those future events, and then generate questions backward from the answers. In this way, future outcomes become supervision signals for both **training** and **evaluation**.

Constructed from more than **700 hours** of Indian suburb videos, YONDER-Bench converts continuous video streams into a large-scale collection of future-grounded interactions. Each benchmark instance contains an observed clip, a hidden future, a visual answer grounded in future evidence, a textual answer describing that evidence, and an associated world operation. The benchmark supports six forms of interaction with predictive simulations: Query, Explain, Inspect, Intervene, Steer, and Plan. Together, these tasks transform future observations into a scalable resource for learning and measuring how language can access, explain, modify, and act upon predictive worlds.

Why Existing Benchmarks Are Not Enough As world models evolve from predictive systems into interactive substrates for robotics, autonomous agents, and physical AI, evaluation must move beyond measuring future prediction alone. Users increasingly seek to query future outcomes, explain predictions, inspect uncertainty, generate counterfactuals, steer simulated futures, and derive actions from imagined trajectories. YONDER-Bench is designed around these interaction capabilities, providing a unified framework for training and evaluating language-guided interaction with predictive simulations through six world operations: Query, Explain, Inspect, Intervene, Steer, and Plan.

3.1 Design Principles

YONDER-Bench follows four design principles. **Future-First Construction:** benchmark items are generated from observed future outcomes rather than speculative questions. We first mine events in the hidden future, extract their visual and textual answers, and only then generate questions grounded in

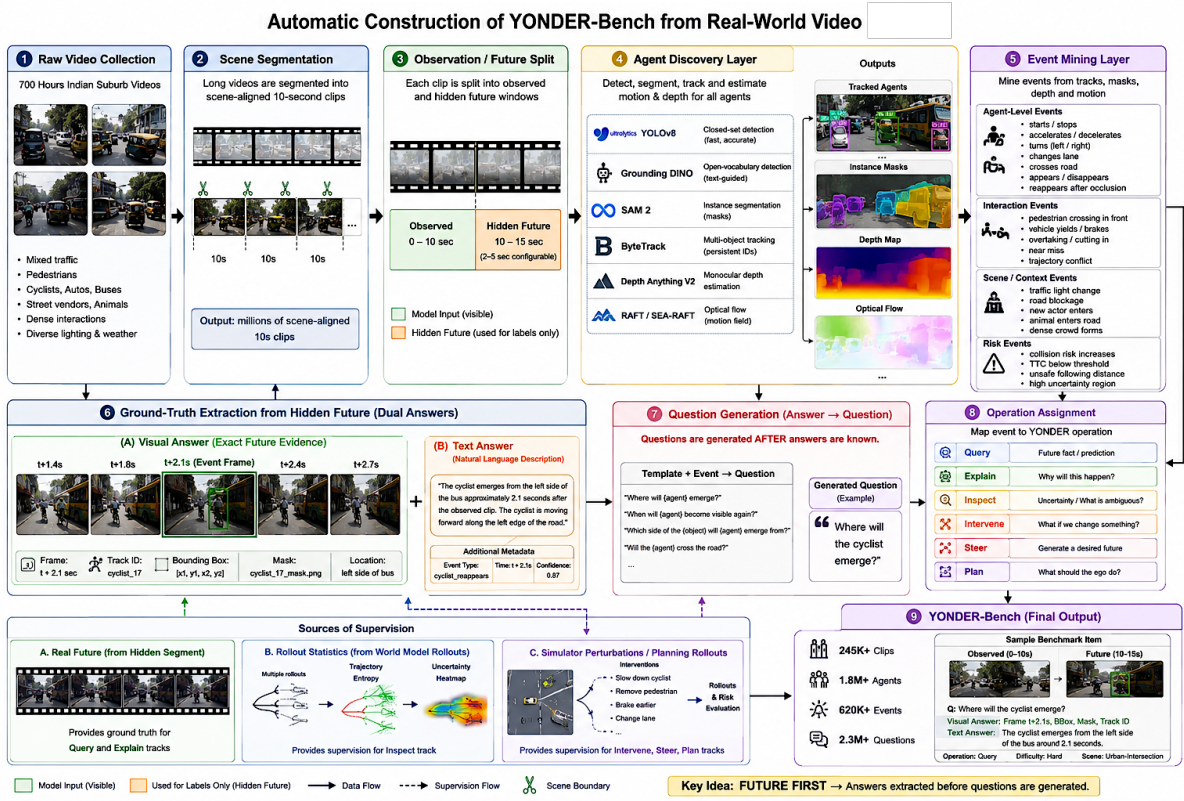


Figure 3: **Automatic construction of YONDER-Bench from real-world video.** Starting from more than **700 hours of Indian suburb videos**, we first perform scene segmentation to obtain scene-consistent clips, each of which is divided into an **observed segment** (visible to the model) and a **hidden future segment** (used only for supervision). An **Agent Discovery Layer** combines object detection, segmentation, tracking, depth estimation, and motion analysis to recover agents, trajectories, masks, depth maps, and optical flow, while an **Event Mining Layer** automatically identifies agent-level, interaction-level, scene-level, and risk-related events. For every detected event, YONDER extracts **dual ground-truth answers** directly from the hidden future: a **visual answer** containing the exact future frame, track identity, bounding box, mask, temporal location, and spatial evidence, and a **text answer** that verbalizes the same future evidence in natural language. Crucially, questions are generated **after** answers are known, ensuring that every benchmark item is grounded in observable future outcomes. Events are subsequently mapped to one of six **world operations**—Query, Explain, Inspect, Intervene, Steer, and Plan. Supervision originates from three complementary sources: **real future observations** for Query and Explain tasks, **world-model rollout statistics** (e.g., trajectory entropy, disagreement, and uncertainty maps) for Inspect tasks, and **simulator perturbations and planning rollouts** for Intervene, Steer, and Plan tasks. The resulting benchmark contains paired observations, future evidence, visual answers, textual answers, operation labels, and evaluation metadata. The central principle of YONDER-Bench is **Future First**: answers are extracted from future evidence before questions are generated, guaranteeing grounded, verifiable, and future-conditioned evaluation of language-guided interaction with predictive simulations.

those answers. **Dual Ground Truth**: every item contains both a **visual answer**—future frame, track, mask, bounding box, timestamp, region, or rollout state—and a **textual answer** that verbalizes the same evidence. **Event-Centric Generation**: question generation is driven by automatically detected motion, interaction, occlusion, scene, risk, uncertainty, counterfactual, steering, and planning events. **Operation-Centric Evaluation**: each item is mapped to one of six world operations—Query, Explain, Inspect, Intervene, Steer, and Plan—so evaluation measures language-guided interaction with predictive simulations rather than passive video understanding.

3.2 Automatic Construction Pipeline

Figure 3 gives the full **future-first** construction pipeline, and Table 1 details each processing stage. Starting from more than **700 hours** of Indian suburb videos, we segment long-form recordings into scene-consistent clips and split each clip into a **10-second observed window** and a **2-5 second hidden future**. The observed window is shown to the model; the hidden future is used to mine events, extract answers, and construct supervision.

Table 1: **Automatic construction pipeline of YONDER-Bench.** Starting from long-form real-world videos, YONDER-Bench automatically discovers agents, mines future events, extracts dual ground-truth answers, and generates future-grounded questions. Importantly, questions are generated *after* answers are extracted from hidden future observations, ensuring that every benchmark item remains verifiable and grounded in observable evidence.

Stage	Input	Methods / Tools	Output	Benchmark Usage
Scene Segmentation	Long-form videos (>700h)	Scene-boundary detection, temporal splitting, duplicate removal	Scene-consistent 10-second clips	Defines benchmark units and observation windows
Observation-Future Split	10-second clips	Temporal partitioning	Observed segment (0–10s) + Hidden Future (10–15s)	Observed segment is visible to models; hidden future is reserved for supervision
Agent Discovery	Observed + future frames	YOLOv8, GroundingDINO, SAM 2	Agent classes, bounding boxes, masks	Identifies pedestrians, cyclists, vehicles, animals, vendors, and other scene entities
Multi-Object Tracking	Detected agents	ByteTrack / BoT-SORT	Persistent agent identities and trajectories	Supports motion analysis, interactions, and future localization
Motion Estimation	Video frames	RAFT / SEA-RAFT optical flow	Velocity, heading, acceleration, motion fields	Used for motion-event detection and interaction reasoning
Geometry Estimation	Video frames	Depth Anything V2	Monocular depth maps and relative geometry	Used for distance estimation, occlusion reasoning, and risk computation
Event Mining	Tracks, masks, depth, flow	Rule-based event extraction and trajectory analysis	Motion events, interaction events, scene events, risk events	Creates future-grounded supervision targets
Visual Answer Extraction	Hidden future segment	Future-frame localization and evidence extraction	Future frame, timestamp, track ID, mask, bounding box, region	Produces exact visual ground-truth answers
Text Answer Generation	Visual answer + event metadata	Template generation + LLM paraphrasing	Natural-language answer	Produces textual ground-truth answers aligned with future evidence
Question Generation	Event labels + dual answers	Event-conditioned templates and paraphrasing	Future-grounded questions	Ensures every question has a verifiable answer
Operation Assignment	Question + event type	Rule-based mapping	Query, Explain, Inspect, Intervene, Steer, Plan	Assigns benchmark tracks and evaluation protocols
Supervision Sources	Future observations, rollouts, interventions	Future evidence, entropy analysis, simulator perturbation	Operation-specific labels	Query/Explain from real future, Inspect from uncertainty statistics, Intervene/Steer/Plan from simulator rollouts

The pipeline first builds an agent-level representation of each clip. We combine object detection, open-vocabulary detection, segmentation, multi-object tracking, optical flow, and monocular depth estimation to recover dynamic agents, masks, trajectories, visibility changes, motion statistics, and relative geometry. As summarized in Table 1, this produces persistent tracks for pedestrians, cyclists, motorcycles, buses, auto-rickshaws, animals, vendors, and dense mixed-traffic actors.

Given these tracks and spatial signals, an event-mining layer detects what actually happens in the

hidden future: motion changes, crossings, overtakes, occlusions, reappearances, risk events, scene changes, and interaction patterns. These mined future events are the anchor points of YONDER-Bench. We do not generate questions first. Instead, we first identify an observable future event and extract its answer.

Table 2: **Event taxonomy for automatic construction of YONDER-Bench.** Future events are mined directly from hidden future observations using agent trajectories, occupancy dynamics, depth, motion fields, and interaction statistics. Each event type serves as a source of grounded supervision for generating future-oriented questions, visual answers, and textual answers. Events are grouped into six categories corresponding to increasingly complex forms of predictive reasoning.

Category	Event Type	Detection Signals	Example Question	Primary Operations
Motion	Stop, Start, Accelerate, Decelerate, Turn, Lane Change	Velocity, acceleration, heading angle, trajectory curvature, optical flow	Will the cyclist stop? Which direction will the vehicle turn? Will the motorcycle accelerate?	Query, Explain
Trajectory	Crossing, Merging, Diverging, Overtaking, Re-routing	Future trajectory intersections, lane occupancy, path deviation	Will the pedestrian cross the road? Which vehicle reaches the junction first? Will the motorcycle overtake the bus?	Query, Plan
Interaction	Yielding, Blocking, Following, Passing, Near-Miss	Relative distance, relative velocity, time-to-collision (TTC), trajectory conflict	Why will the vehicle slow down? Which agent influences the cyclist most? Will the pedestrian force the vehicle to yield?	Explain, Plan
Occlusion	Disappear, Reappear, Occluded Crossing, Hidden Actor Emergence	Visibility ratio, mask area reduction, track recovery, depth ordering	Where will the cyclist emerge? Which actor is hidden behind the bus? When will the pedestrian become visible again?	Query, Inspect
Scene	Road Blockage, Crowd Formation, Vehicle Entry, Traffic-Light Change, Animal Entry	Scene occupancy changes, semantic-state transitions, agent appearance	Will the road become blocked? What new agent enters the scene? How does the traffic state change?	Query, Explain
Risk	Collision Risk, Unsafe Proximity, Trajectory Conflict, TTC Violation	Occupancy overlap, minimum distance, collision probability, TTC threshold	Which region is most risky? Which actor contributes most risk? Is a collision likely?	Inspect, Plan
Uncertainty	Trajectory Ambiguity, Multi-Modal Future, Prediction Disagreement, Occlusion Uncertainty	Trajectory entropy, rollout variance, occupancy entropy, model disagreement	Which actor is most uncertain? Why is the prediction ambiguous? Which future branch is most likely?	Inspect
Counterfactual	Speed Change, Agent Removal, Actor Insertion, Path Modification	Simulator perturbation, trajectory intervention, rollout comparison	What if the cyclist brakes? What if the pedestrian does not cross? What if the vehicle accelerates?	Intervene
Steering	Safety-Constrained Future, Conservative Future, Minimum-Risk Future	Optimization objective, rollout scoring, uncertainty minimization	Generate the safest future. Produce the lowest-risk trajectory. Reduce uncertainty.	Steer
Planning	Action Selection, Route Choice, Yield Decision, Brake/Go Decision	Candidate rollout evaluation, risk-aware planning objectives	What should the vehicle do? Should the robot wait or move? Which action minimizes risk?	Plan

For each event, YONDER-Bench stores **dual ground truth**. The **visual answer** records the exact future evidence—timestamp, frame, track ID, bounding box, mask, region, trajectory, or rollout state. The **textual answer** verbalizes the same evidence in natural language. Only after both answers are available do we generate an event-conditioned question and assign it to a world operation. This gives each benchmark item a complete tuple:

$$(C_{\text{obs}}, C_{\text{fut}}, q, a_{\text{vis}}, a_{\text{text}}, o, e, m),$$

where C_{obs} is the observed clip, C_{fut} is the hidden future, q is the question, a_{vis} and a_{text} are the visual and textual answers, o is the world operation, e is the mined event, and m is the evaluation metric.

This construction makes the benchmark scalable and auditable: every question is generated **after**

its answer is known, every answer is tied to future evidence, and every item can be traced back to an observed event in the hidden future.

3.3 Event Taxonomy

The foundation of YONDER-Bench is its event taxonomy, which transforms continuous video streams into structured future interactions. As summarized in Table 2, events are automatically mined from trajectories, visibility patterns, motion fields, occupancy dynamics, uncertainty estimates, and simulator rollouts. Each event serves as a source of future-grounded supervision from which visual answers, textual answers, and benchmark questions are derived.

At the most fundamental level, YONDER-Bench captures **motion events** such as starting, stopping, accelerating, braking, turning, lane changes, merging, and crossings. These events are detected directly from future trajectories and provide supervision for future-state retrieval and temporal reasoning. Building upon individual motion, the benchmark identifies **trajectory and interaction events** including overtaking, yielding, following, blocking, passing, and near-miss encounters. Such events emerge from relationships between multiple agents and are critical for explanation, planning, and multi-agent reasoning.

A distinctive aspect of YONDER-Bench is its emphasis on **occlusion events**. Dense real-world scenes frequently contain hidden actors, temporary disappearances, reappearances, and occluded crossings. These events are especially valuable because they reveal the uncertainty, ambiguity, and latent expectations that predictive world models must maintain when observations are incomplete.

Beyond agent-centric behavior, the benchmark captures **scene events** and **risk events**. Scene events include crowd formation, road blockage, traffic-state transitions, vehicle entry from side roads, and animal intrusion. Risk events arise from occupancy conflicts, unsafe proximity, low time-to-collision situations, and potential collision trajectories. Together, these events connect future prediction with safety-aware reasoning and decision making.

YONDER-Bench further extends beyond observed futures through **uncertainty**, **counterfactual**, **steering**, and **planning events**. Uncertainty events are derived from rollout disagreement, trajectory entropy, and occupancy ambiguity. Counterfactual events are generated through controlled interventions such as speed modification, path alteration, actor insertion, or actor removal. Steering events arise from optimization objectives such as safety, caution, or uncertainty reduction, while planning events are generated from evaluating alternative actions across candidate futures. Collectively, these event categories span the full spectrum of language-guided interaction with predictive simulations, from retrieving future outcomes to inspecting uncertainty, generating alternative worlds, steering behavior, and selecting actions.

3.4 Task Taxonomy

The event taxonomy defines *what happens* in the future; the task taxonomy defines *how a model must interact* with that future. As shown in Table 3, YONDER-Bench organizes evaluation into six operation-level tracks: Query, Explain, Inspect, Intervene, Steer, and Plan. Each track is associated with a distinct supervision source, answer format, and evaluation protocol.

Query measures whether a model can retrieve future-state information from predictive simulations, such as agent position, motion direction, crossing outcome, occupancy region, or event timing. **Explain** measures whether the model can connect a predicted future to supporting evidence, such as a pedestrian entering a lane, a vehicle slowing, an occluded actor reappearing, or a trajectory conflict emerging. **Inspect** measures whether the model can expose uncertainty, ambiguity, hidden actors, multimodal futures, and belief-like assumptions encoded in the predictive state.

The remaining tracks evaluate whether language can modify or use predictive futures. **Intervene** tests counterfactual reasoning by altering actors, motion, object states, or scene conditions and comparing the resulting rollout. **Steer** evaluates objective-conditioned simulation, where futures are generated under constraints such as safety, caution, minimum risk, or uncertainty reduction. **Plan** evaluates action selection by comparing candidate rollouts and selecting actions that optimize task-specific utility.

Table 3: **YONDER-Bench task taxonomy.** Unlike conventional video understanding benchmarks that evaluate recognition or description of observed content, YONDER-Bench evaluates language-guided interaction with predictive simulations. Each task is associated with a distinct world operation, supervision source, answer format, and evaluation protocol. Query and Explain tasks are grounded in real future observations, Inspect tasks are grounded in uncertainty and rollout statistics, and Intervene, Steer, and Plan tasks are grounded in simulator-generated futures.

Track	World Operation	Example Question	Ground Truth Source	Answer Format	Evaluation Metrics
Query	Future-state retrieval	Where will the cyclist emerge? Will the pedestrian cross? Which vehicle reaches the junction first?	Hidden future observations	Visual answer + Text answer	Accuracy, ADE/FDE, IoU, Event F1, Temporal Error
Explain	Prediction explanation	Why will the car slow down? Why is the cyclist turning? What causes the vehicle to yield?	Future events + causal evidence	Text answer + Supporting evidence	Cause Accuracy, Evidence Localization, LLM-Judge
Inspect	Uncertainty and belief inspection	Which actor is most uncertain? Which region contributes most ambiguity? Why is the prediction uncertain?	Trajectory entropy, occupancy entropy, rollout disagreement	Text answer + Uncertainty map	Rank Correlation, Top- k Accuracy, Calibration Error
Intervene	Counterfactual simulation	What if the cyclist brakes? What if the pedestrian does not cross? What if the vehicle accelerates?	Simulator perturbation	Counterfactual rollout	Trajectory Consistency, Counterfactual Validity, Risk Delta
Steer	Objective-conditioned simulation	Generate the safest future. Produce a conservative trajectory. Minimize collision risk.	Optimization-guided rollouts	Steered rollout	Risk Reduction, Collision Rate, Uncertainty Reduction
Plan	Action selection and decision making	What should the vehicle do? Should the robot wait or move? Which action minimizes risk?	Candidate rollout evaluation	Action recommendation + Justification	Success Rate, Utility Score, Planning Accuracy

Together, these six tasks move evaluation beyond asking whether a model can predict what happens next. They ask whether a model can make simulated futures **retrievable, explainable, inspectable, alterable, steerable, and actionable** through language.

3.5 Dataset Statistics

Table 4 summarizes the scale and composition of YONDER-Bench across video sources, agent tracks, mined events, simulation rollouts, task instances, dual answers, and evaluation splits. The benchmark is built from more than **700 hours** of real-world Indian suburb videos, segmented into scene-consistent clips with a **10-second observed window** and a **2-5 second hidden future**. This structure allows each instance to pair what the model observes with future evidence used for supervision.

Automatic agent discovery yields persistent tracks for pedestrians, cyclists, vehicles, motorcycles, animals, vendors, carts, and other dynamic entities. Event mining converts these tracks into motion, interaction, occlusion, scene, and risk events, while simulator perturbations produce counterfactual, steered, and planning rollouts. Each item is paired with **dual ground truth**: a visual answer grounded in future frames, tracks, masks, regions, or rollout states, and a textual answer verbalizing the same evidence.

We construct train, validation, and test splits at the **scene level** rather than the clip level to avoid temporal leakage and near-duplicate overlap across splits. The resulting dataset provides large-scale supervision for both training and evaluating language-guided interaction with predictive simulations while preserving direct grounding in observable future evidence.

3.6 Evaluation Protocol

YONDER-Bench evaluates whether predictive simulations can be accessed, interpreted, modified, and utilized through language. Because the six world operations rely on different forms of supervision, each track is associated with a distinct evaluation protocol, as summarized in Table 3.

Query tasks evaluate future-state retrieval. Models must recover information contained within predictive futures, including agent locations, trajectories, event outcomes, occupancies, and temporal

Table 4: **YONDER-Bench dataset statistics.** YONDER-Bench is constructed from **700+ hours** of Indian suburb videos and organized into paired **10-second observed clips** and **2–5 second hidden futures**. Each benchmark instance contains future evidence, a visual answer, a textual answer, an operation label, and evaluation metadata.

Group	Statistic	Count	Description
Video Source	Raw duration	700+ hours	Long-form Indian suburb videos containing mixed traffic, pedestrians, cyclists, buses, motorcycles, animals, vendors, occlusions, and dense interactions.
	Observed window	10 sec	Clip segment visible to the evaluated model.
	Hidden future	2–5 sec	Future segment withheld from the model and used for answer extraction.
	Scene-consistent clips	N_{clips}	Automatically segmented clips after scene-boundary detection and quality filtering.
Agent Layer	Tracked agents	N_{agents}	Persistent identities for pedestrians, cyclists, vehicles, motorcycles, animals, vendors, carts, and other dynamic entities.
	Agent tracks	N_{tracks}	Frame-level trajectories with bounding boxes, masks, velocity, heading, and visibility status.
	Occluded tracks	$N_{\text{occ-track}}$	Tracks containing disappearance, partial visibility, or reappearance events.
Event Layer	Motion events	N_{motion}	Start, stop, accelerate, decelerate, turn, merge, lane-change, and crossing events.
	Interaction events	$N_{\text{interaction}}$	Yielding, overtaking, blocking, following, near-miss, and multi-agent conflict events.
	Occlusion events	$N_{\text{occlusion}}$	Hidden actor emergence, disappearance, reappearance, and occluded crossing events.
	Scene events	N_{scene}	Traffic-state changes, road blockage, crowd formation, side-lane entry, and animal-entry events.
	Risk events	N_{risk}	Unsafe proximity, low time-to-collision, occupancy overlap, and trajectory conflict events.
Simulation Layer	Counterfactual rollouts	N_{cf}	Intervention-generated futures such as braking, stopping, removal, insertion, or path modification.
	Steered rollouts	N_{steer}	Objective-conditioned futures such as safest, conservative, minimum-risk, or uncertainty-reduced rollouts.
	Planning rollouts	N_{plan}	Candidate action rollouts used to evaluate yield, stop, slow, continue, or reroute decisions.
Benchmark Tasks	Query	N_{query}	Future-state retrieval questions grounded in hidden future observations.
	Explain	N_{explain}	Causal and evidence-based questions grounded in detected future events.
	Inspect	N_{inspect}	Uncertainty, ambiguity, hidden-state, and belief-inspection questions grounded in rollout statistics.
	Intervene	$N_{\text{intervene}}$	Counterfactual questions grounded in simulator perturbations.
	Steer	$N_{\text{steer-task}}$	Objective-conditioned simulation questions grounded in optimized rollouts.
	Plan	$N_{\text{plan-task}}$	Action-selection questions grounded in candidate rollout evaluation.
Answer Layer	Visual answers	N_{visual}	Future frame, timestamp, track ID, mask, bounding box, spatial region, or rollout state.
	Text answers	N_{text}	Natural-language answers verbalizing the same future evidence.
	Dual-answer items	N_{dual}	Benchmark items containing both visual and textual ground truth.
Evaluation Split	Train / validation / test	$N_{\text{train}}/N_{\text{val}}/N_{\text{test}}$	Scene-disjoint split to prevent near-duplicate clips from appearing across splits.
	Total instances	N_{total}	All benchmark items across Query, Explain, Inspect, Intervene, Steer, and Plan.

events. Evaluation combines event accuracy, localization accuracy, temporal error, occupancy overlap, and trajectory forecasting metrics such as Average Displacement Error (ADE) and Final Displacement Error (FDE).

Explain tasks evaluate whether predictions can be linked to supporting evidence. Models are required to identify the causes underlying future events and justify their predictions using relevant actors, interactions, or scene changes. Evaluation measures causal correctness, evidence localization, and explanation quality through both automatic metrics and LLM-assisted assessment.

Inspect tasks evaluate uncertainty awareness and introspection. Models must identify ambiguous regions, hidden actors, multimodal futures, rollout disagreement, and other sources of predictive uncertainty. Evaluation uses calibration metrics, entropy alignment, rank correlation, uncertainty localization, and identification accuracy for uncertain agents or regions.

Intervene tasks evaluate counterfactual reasoning. Given modifications to agents, trajectories, or scene conditions, models must predict how the future changes under intervention. Evaluation compares generated counterfactual futures against simulator rollouts using trajectory consistency, intervention validity, and outcome preservation metrics.

Steer tasks evaluate objective-conditioned simulation. Models are asked to generate futures satisfying constraints such as safety, caution, efficiency, or uncertainty reduction. Evaluation measures objective satisfaction through collision-rate reduction, risk reduction, uncertainty reduction, and rollout quality under specified constraints.

Plan tasks evaluate decision making over predictive futures. Models must compare candidate actions, reason over alternative outcomes, and select actions that optimize task-specific objectives. Evaluation uses planning success rate, utility-based scoring, agreement with minimum-risk actions, and downstream task performance.

Collectively, these protocols measure whether a model can make predictive simulations **retrievable, explainable, inspectable, counterfactual, steerable, and actionable** through language. Rather than evaluating future prediction in isolation, YONDER-Bench evaluates a model's ability to interact with future possibilities.

Algorithm 1 Future-First Construction of YONDER-Bench

Require: Raw video collection $\mathcal{V}$, observation length T_{obs} , future horizon T_{fut}

Ensure: YONDER-Bench dataset $\mathcal{D}$ with questions, visual answers, text answers, operations, and metadata

```
1:  $\mathcal{D} \leftarrow \emptyset$ 
2: for each long video  $V \in \mathcal{V}$  do
3:   Segment  $V$  into scene-consistent clips:
       $\mathcal{C} \leftarrow \text{SceneSegment}(V)$ 

4:   for each clip  $C \in \mathcal{C}$  do
5:     Split  $C$  into observed and hidden future windows:
       $C_{\text{obs}} = C[0 : T_{\text{obs}}], \quad C_{\text{fut}} = C[T_{\text{obs}} : T_{\text{obs}} + T_{\text{fut}}]$ 

6:     Detect agents in observed and future frames:
       $\mathcal{B}, \mathcal{M} \leftarrow \text{DetectSegment}(C_{\text{obs}}, C_{\text{fut}})$ 
      ▷ YOLO / GroundingDINO + SAM 2

7:     Track agents across time:
       $\mathcal{T} \leftarrow \text{Track}(\mathcal{B}, \mathcal{M})$ 
      ▷ ByteTrack / BoT-SORT

8:     Estimate motion and geometry:
       $\mathcal{F}, \mathcal{G} \leftarrow \text{FlowDepth}(C_{\text{obs}}, C_{\text{fut}})$ 
      ▷ RAFT / SEA-RAFT + Depth Anything V2

9:     Mine future events from tracks, masks, flow, and depth:
       $\mathcal{E} \leftarrow \text{MineEvents}(\mathcal{T}, \mathcal{M}, \mathcal{F}, \mathcal{G})$ 

10:    for each detected event  $e \in \mathcal{E}$  do
11:      Extract visual answer from hidden future:
       $a_{\text{vis}} \leftarrow \text{ExtractVisualAnswer}(e, C_{\text{fut}}, \mathcal{T}, \mathcal{M})$ 
      ▷ future frame, timestamp, track ID, bbox, mask, region

12:      Generate textual answer from event metadata:
       $a_{\text{text}} \leftarrow \text{Verbalize}(e, a_{\text{vis}})$ 

13:      Generate future-grounded question after answer extraction:
       $q \leftarrow \text{GenerateQuestion}(e, a_{\text{vis}}, a_{\text{text}})$ 

14:      Assign world operation:
       $o \leftarrow \text{MapOperation}(e, q)$ 

15:      Select evaluation protocol:
       $m \leftarrow \text{AssignMetric}(o, e)$ 

16:      if  $\text{QualityCheck}(q, a_{\text{vis}}, a_{\text{text}}, e)$  then
17:        Add benchmark item:
       $\mathcal{D} \leftarrow \mathcal{D} \cup \{(C_{\text{obs}}, C_{\text{fut}}, q, a_{\text{vis}}, a_{\text{text}}, o, e, m)\}$ 

18:      end if
19:    end for
20:  end for
21: end for
22: Construct scene-disjoint train/validation/test splits.
23: return  $\mathcal{D}$ 
```

4 Evaluation and Results

YONDER-Bench evaluates whether a model can answer language queries by grounding them in future evidence. Each instance provides a **10-second observed clip** and a question, while the hidden future supplies two targets: a **visual answer** and a **textual answer**.

We evaluate YONDER along three axes. **Future evidence prediction** measures whether the model identifies the correct future frame, event, trajectory, region, or rollout state. **Future description generation** measures whether it verbalizes that evidence accurately. **Grounding consistency** measures whether the textual answer is supported by the predicted visual evidence rather than being merely plausible language.

4.1 Evaluation Setting

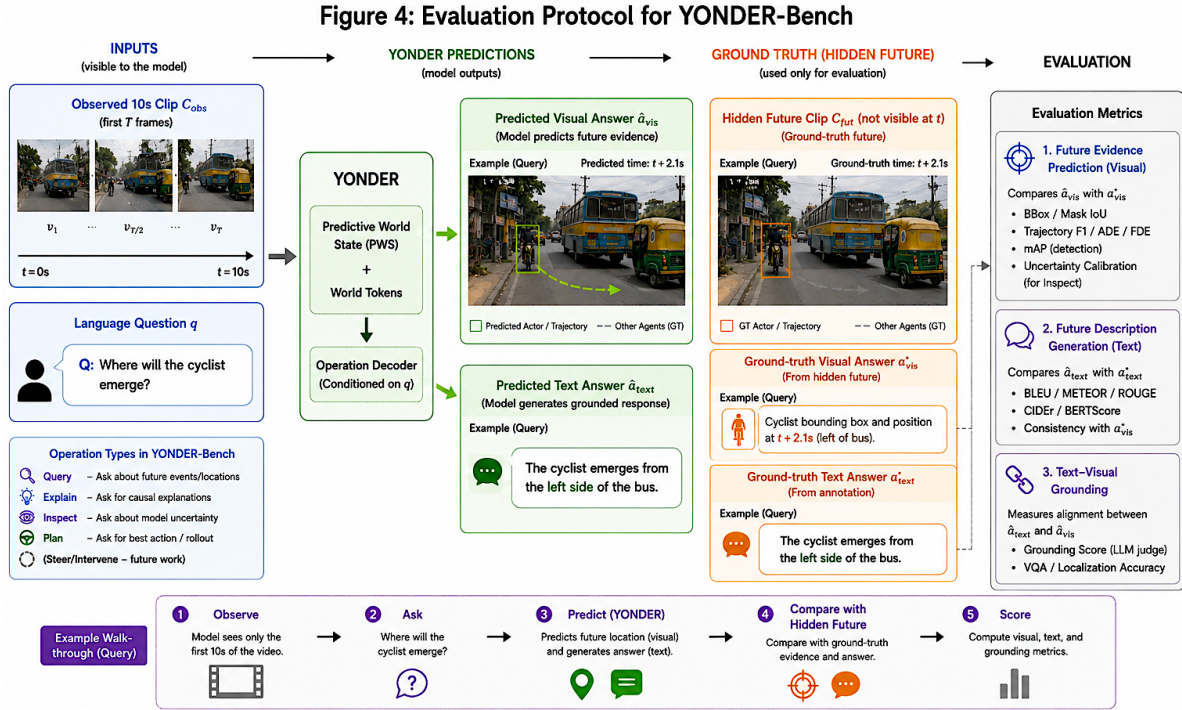
For each benchmark instance, the model receives an observed clip

$$C_{\text{obs}} = \{v_1, v_2, \dots, v_T\}$$

and a natural-language question q . The hidden future

$$C_{\text{fut}} = \{v_{T+1}, v_{T+2}, \dots, v_{T+K}\}$$

is not provided to the model during inference; it is used only to define ground truth.



At inference, YONDER receives only the observed clip C_{obs} and a language question q . The hidden future C_{fut} is never shown to the model and is used solely to construct ground-truth visual (a_{vis}^*) and textual (a_{text}^*) answers. The model outputs a predicted visual answer $\hat{a}_{\text{vis}}$ and a textual answer $\hat{a}_{\text{text}}$. We evaluate future evidence prediction, future description generation, and text-visual grounding to comprehensively measure performance.

Figure 4: **Evaluation protocol for YONDER-Bench.** During inference, the model receives only the observed clip $C_{\text{obs}} = \{v_1, \dots, v_T\}$ and a natural-language question q . YONDER processes the Predictive World State (PWS) through the World-Language Interface and Operation Decoder to produce two aligned outputs: a **future visual answer** $\hat{a}_{\text{vis}}$ (e.g., future frame, actor, trajectory, region, uncertainty map, or rollout state) and a **textual answer** $\hat{a}_{\text{text}}$. The hidden future segment $C_{\text{fut}} = \{v_{T+1}, \dots, v_{T+K}\}$ remains inaccessible to the model and is used exclusively to construct the ground-truth visual answer a_{vis}^* and textual answer a_{text}^* . Evaluation is performed along three complementary dimensions: **Future Evidence Prediction**, which measures whether the correct future evidence is identified; **Future Description Generation**, which measures whether that evidence is described faithfully in natural language; and **Text-Visual Grounding**, which measures consistency between predicted visual and textual outputs.

YONDER predicts two aligned outputs:

$$\hat{a}_{\text{vis}}, \hat{a}_{\text{text}},$$

where $\hat{a}_{\text{vis}}$ is the predicted future visual evidence and $\hat{a}_{\text{text}}$ is the predicted textual answer. The visual answer may specify a future frame, timestamp, track identity, bounding box, segmentation mask, trajectory, occupancy region, uncertainty map, or rollout state. The textual answer verbalizes the same future evidence in natural language.

The ground-truth targets are extracted from the hidden future:

$$a_{\text{vis}}^*, a_{\text{text}}^*.$$

Thus, each evaluation item is represented as

$$\mathcal{E} = (C_{\text{obs}}, q, C_{\text{fut}}, a_{\text{vis}}^*, a_{\text{text}}^*),$$

while the model is evaluated only from

$$(C_{\text{obs}}, q) \rightarrow (\hat{a}_{\text{vis}}, \hat{a}_{\text{text}}).$$

The central evaluation question is therefore: *can the model identify the correct future evidence and describe it faithfully without observing the future during inference?*

4.2 Task Protocols and Metrics

Table 5 summarizes the evaluation protocol. Each YONDER-Bench item requires two aligned predictions: a **future visual answer** and a **textual answer**. The visual answer tests whether the model finds the right future evidence; the textual answer tests whether it describes that evidence faithfully.

Table 5: **YONDER-Bench task protocols and evaluation metrics.** Each task requires predicting a **future visual answer** and a corresponding **textual answer**. Visual metrics evaluate whether the correct future evidence is identified, while textual metrics evaluate whether that evidence is described faithfully. Grounding consistency measures agreement between the two outputs.

Operation	Visual Answer	Text Answer	Evaluation Metrics
Query	Future frame, actor, trajectory, region, event outcome	Future-state description	Frame Accuracy, Temporal Error, IoU, ADE/FDE, Event Accuracy, Event F1
Explain	Supporting actor, interaction, causal region, evidence trajectory	Causal explanation	Evidence Localization, Cause Accuracy, Event F1, LLM-Judge
Inspect	Uncertainty region, hidden actor, ambiguous trajectory, competing future	Uncertainty description	Uncertainty Localization, Rank Correlation, Calibration Error, LLM-Judge
Intervene	Counterfactual future state, modified trajectory, altered event	Counterfactual description	Counterfactual Consistency, ADE/FDE, Event Preservation, Outcome Accuracy
Steer	Objective-conditioned future, safety-aware rollout, risk-reduced trajectory	Steering justification	Collision Reduction, Risk Reduction, Objective Satisfaction, Rollout Quality
Plan	Selected future rollout, action-conditioned future trajectory	Action recommendation	Planning Success Rate, Utility Score, Min-Risk Agreement, Outcome Accuracy

The six operations define different forms of future-grounded answering. **Query** measures retrieval of future events, actors, trajectories, and outcomes. **Explain** measures whether the model identifies evidence responsible for a future event. **Inspect** measures recognition of uncertainty, ambiguity, hidden actors, and multimodal futures. **Intervene** measures counterfactual future prediction under a

specified change. **Steer** measures whether the predicted future satisfies an objective such as safety or reduced risk. **Plan** measures whether the model selects an action justified by future evidence.

We evaluate visual answers using metrics matched to the predicted evidence type: temporal error and frame accuracy for future-frame localization; bounding-box IoU, mask IoU, and occupancy overlap for spatial grounding; ADE and FDE for trajectories; and event accuracy or macro-F1 for event prediction. Textual answers are evaluated using answer accuracy, semantic similarity, event-level F1, and LLM-assisted judgment; explanation tasks additionally use causal correctness and evidence attribution.

Finally, we report **text-visual grounding consistency**, which measures whether the textual answer is supported by the predicted visual evidence. This penalizes fluent but unsupported answers and separates genuine future-grounded reasoning from plausible language generation.

4.3 Future Evidence Prediction

Table ?? reports how accurately YONDER predicts the **visual answer** associated with each question. This evaluation asks whether the model can localize the relevant future evidence before producing language: the correct future frame, agent, trajectory, region, event, uncertainty map, or rollout state.

Performance is strongest on Query tasks, where the target evidence is usually a directly observable future event or trajectory. Explain requires a harder form of localization because the model must identify not only what happens, but which actor, region, or interaction supports the predicted outcome. Inspect is more challenging still, as the target often corresponds to ambiguity, occlusion, trajectory entropy, or disagreement across possible futures.

The write-oriented tasks—Intervene, Steer, and Plan—test whether YONDER can predict or select future evidence under modified conditions. These tasks are harder because success depends on comparing alternative rollouts rather than retrieving a single observed future. Overall, the visual-answer results measure the core capability required by YONDER: **finding the right future before describing it**.

Table 6: **Main results on YONDER-Bench.** Results are organized into three complementary dimensions: **Future Evidence Prediction**, which evaluates whether YONDER identifies the correct future evidence; **Future Description Generation**, which evaluates whether that evidence is described accurately in natural language; and **Grounding Analysis**, which evaluates whether textual answers remain faithful to the predicted future evidence. Higher is better for all metrics except Hallucination Rate.

Operation	Future Evidence Prediction				Future Description Generation			Grounding Analysis	
	Frame	Event	Trajectory	Region	Answer	Semantic	Judge	Consistency	Halluc.
Query	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Explain	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Inspect	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Intervene	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Steer	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Plan	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Average	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD

4.4 Future Description Generation

Table ?? reports how accurately YONDER predicts the **textual answer** associated with each question. This evaluation asks whether the model can convert future evidence into a faithful natural-language response, rather than producing a plausible but unsupported answer.

Performance is strongest when the visual evidence is localized and unambiguous, as in Query and many Explain examples. Tasks such as Inspect, Steer, and Plan are more linguistically variable because the answer may involve uncertainty, objectives, alternatives, or action justification. For these cases, exact-match scores are less informative than semantic similarity, event-level correctness, and evidence-aware judgment.

Overall, the text-answer results evaluate whether YONDER can translate predictive simulations into precise, grounded, and readable language.

4.5 Grounding Analysis

Table ?? evaluates **text-visual grounding consistency**. This measures whether the textual answer is supported by the predicted visual answer: the referenced actor, event, location, time, trajectory, uncertainty region, or rollout state should match the future evidence identified by the model.

This analysis is critical because future-oriented language can sound plausible even when it is not grounded. A response such as *the cyclist emerges from the left* is correct only if the predicted visual evidence also localizes the cyclist on the left side at the relevant future time. We therefore evaluate not only whether the answer is fluent, but whether it is anchored to the same future evidence used for visual prediction.

High grounding consistency indicates that YONDER is not merely generating likely descriptions of traffic scenes; it is verbalizing the predictive evidence it has selected. This separates **future-grounded reasoning** from generic future narration.

Table 7: **Ablation study of YONDER.** We evaluate the contribution of the principal components of YONDER across the six benchmark operations. Results are reported using the overall task score defined in Section ?. Grouped ablations isolate the impact of the **Predictive World State**, **World-Language Interface**, and **Operation Layer**. Higher is better.

Variant	Query	Explain	Inspect	Intervene	Steer	Plan	Avg.
Predictive World State Ablations							
Full YONDER	TBD	TBD	TBD	TBD	TBD	TBD	TBD
No-PWS	TBD	TBD	TBD	TBD	TBD	TBD	TBD
No-U (Uncertainty)	TBD	TBD	TBD	TBD	TBD	TBD	TBD
No-R (Rollouts)	TBD	TBD	TBD	TBD	TBD	TBD	TBD
No-A (Saliency Regions)	TBD	TBD	TBD	TBD	TBD	TBD	TBD
World-Language Interface Ablations							
No-WLI	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Mean Pooling (No Resampler)	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Operation Layer Ablations							
No-Operation Decoder	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Unified Prompting	TBD	TBD	TBD	TBD	TBD	TBD	TBD
No-Control Head	TBD	TBD	TBD	TBD	TBD	TBD	TBD

4.6 Ablation Study

Table 7 isolates the contribution of each YONDER component. We report ablations along the same three axes used in the main evaluation: **future evidence prediction**, **future description generation**, and **text-visual grounding consistency**.

Removing the **Predictive World State** causes the largest drop, indicating that future-grounded answering depends on structured access to predicted latents, trajectories, occupancy, uncertainty, and rollouts. Removing uncertainty fields mainly degrades Inspect, while removing rollout tokens primarily affects Intervene, Steer, and Plan. Disabling the operation decoder weakens both visual and textual performance, showing that explicit operation grounding is important even when the final output is language.

These ablations support the central design of YONDER: accurate future-grounded answering requires not only a world model, but a structured predictive state, a language interface, and operation-aware access to future evidence.

4.7 Qualitative Analysis

Figure 5 shows representative examples across the six YONDER-Bench tracks. Each example includes the observed clip, the question, the predicted visual answer, the ground-truth future evidence, and the generated textual answer.

Operation & Question	Observed 10s Clip (t) What the model sees	Hidden Future / GT Evidence (t + k) Ground-truth future not visible at t	YONDER Predicted Future Evidence Predicted by our model	Text Answer (Grounded) Answer grounded in future evidence
QUERY Q: Where will the cyclist emerge? Video length: 10s	t (last frame)	t + 2.1 s	t + 2.1 s (pred)	✓ The cyclist emerges from the left side of the bus. Grounding Score: 0.92
EXPLAIN Q: Why will the vehicle slow down?	t (last frame)	t + 1.6 s	t + 1.6 s (pred)	✓ The vehicle slows because the pedestrian enters its path. Grounding Score: 0.93
INSPECT Q: Which region is most uncertain?	t (last frame)	t + 2.3 s (GT future)	t + 2.3 s (pred) – Uncertainty Heatmap	⚙️ The region behind the parked truck has the highest uncertainty. Grounding Score: 0.89
PLAN Q: What should the ego vehicle do?	t (last frame)	t + 2.6 s (GT future)	t + 2.6 s (pred) – Selected Rollout	✓ Slow down and yield before entering the intersection. Grounding Score: 0.94
— GT Actor / Trajectory — Predicted Actor / Trajectory — — Other Agents (GT) Low — High Uncertainty (Pred) Risk Zone (GT) Safe Rollout (Pred)				

Figure 5: Qualitative examples from YONDER-Bench. Each row shows one operation type and one question. The model observes only the 10-second clip in column 2. Columns 3–4 show the ground-truth future evidence and the future evidence predicted by YONDER. Column 5 presents the language answer grounded in the predicted future evidence with a grounding score.

Figure 5: **Qualitative examples from YONDER-Bench.** Each row shows one language-guided interaction with a predictive simulation. The model receives only the **observed 10-second clip** and a natural-language question. The **hidden future** provides ground-truth evidence but is not visible during inference. YONDER predicts the relevant **future evidence**—such as an actor trajectory, bounding box, uncertainty region, risk zone, or selected rollout—and then generates a **textual answer** grounded in that evidence. The examples illustrate Query, Explain, Inspect, and Plan operations in Indian suburb traffic scenes.

The qualitative examples highlight three recurring behaviors. First, YONDER can localize future evidence under occlusion, such as identifying where a hidden actor reappears. Second, it can distinguish confident predictions from ambiguous futures by pointing to uncertainty regions or competing rollouts. Third, for intervention, steering, and planning tasks, it can compare alternative future states and produce action-oriented descriptions grounded in the selected rollout.

Overall, the qualitative results illustrate the main goal of YONDER: converting predictive simulations into future evidence that can be located, described, inspected, and acted upon through language.

5 Conclusion

We introduced **YONDER**, a framework that makes predictive world models **language-addressable**. By organizing simulated futures into a structured Predictive World State and exposing them through executable world operations, YONDER enables users to query, explain, inspect, intervene, steer, and plan over imagined futures rather than merely observe rollouts.

We also introduced **YONDER-Bench**, a **future-first** benchmark built from real video futures. Instead of writing questions first, YONDER-Bench extracts visual and textual answers from hidden future evidence and then generates questions grounded in those answers. This yields verifiable supervision for evaluating both future evidence prediction and future-grounded language generation.

Together, YONDER and YONDER-Bench shift world-model evaluation from *Can the model predict*

what happens next? to *Can humans interact with what the model predicts?* This reframes world models from passive simulators into interactive reasoning systems for robotics, autonomous agents, and physical AI.

References

- Niket Agarwal et al. 2025. [Cosmos world foundation model platform for physical ai](#). *arXiv preprint arXiv:2501.03575*.
- Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. 2025. [V-jepa 2: Self-supervised video models enable understanding, prediction and planning](#). *arXiv preprint arXiv:2506.09985*.
- Adrien Bardes, Quentin Garrido, Jean Ponce, Yann LeCun, Michael Rabbat, and Nicolas Ballas. 2024. [Revisiting feature prediction for learning visual representations from video](#). *arXiv preprint arXiv:2404.08471*.
- Jake Bruce et al. 2024. [Genie: Generative interactive environments](#). *arXiv preprint arXiv:2402.15391*.
- David Ha and Jürgen Schmidhuber. 2018. [World models](#). *arXiv preprint arXiv:1803.10122*.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. 2023. [Mastering diverse domains through world models](#). In *NeurIPS*.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. [Lora: Low-rank adaptation of large language models](#). *arXiv preprint arXiv:2106.09685*.
- Yann LeCun. 2022. [A path towards autonomous machine intelligence](#). OpenReview.
- Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023a. [Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models](#). In *International Conference on Machine Learning*.
- KunChang Li et al. 2023b. [Videochat: Chat-centric video understanding](#). *arXiv preprint arXiv:2305.06355*.
- Muhammad Maaz, Hanoona Rasheed, Salman Khan, and Fahad Shahbaz Khan. 2023. [Video-chatgpt: Towards detailed video understanding via large vision and language models](#). *arXiv preprint arXiv:2306.05424*.